



Dependency Risk Analyzer

Stop guessing. Know exactly which open-source projects are safe to depend on — backed by data, not stars.

OPEN SOURCE INTELLIGENCE

GITHUB ANALYTICS



The Problem: Stars Don't Equal Safety

100K Stars

Could be completely abandoned

5K Stars

Could be actively maintained

No Easy Way to Tell

Current signals are misleading

Four Real Signals That Matter



Activity

Are they pushing code regularly?



Team Size

Is it one person or a team?



Response Speed

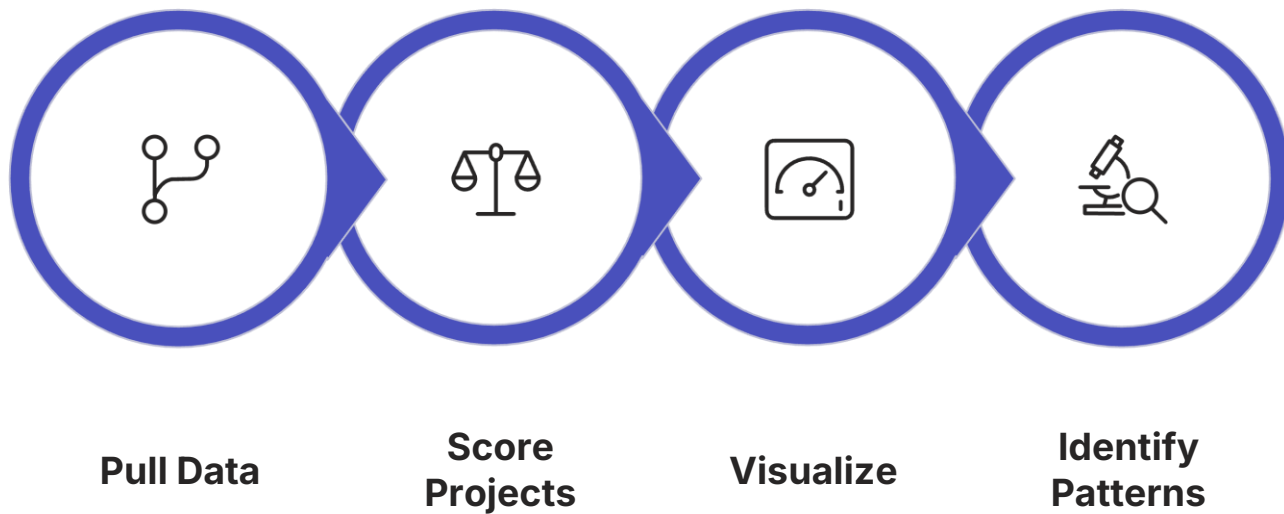
How fast are bugs fixed?



Consistency

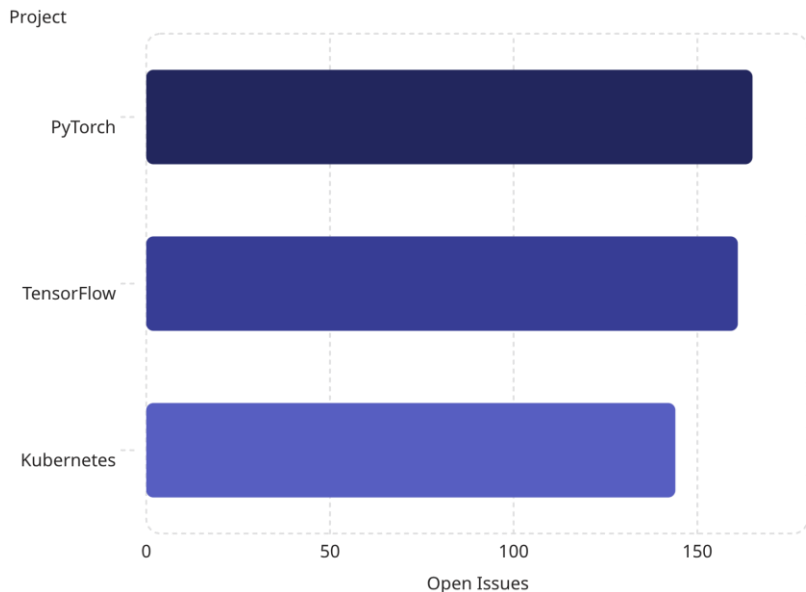
Are updates regular over time?

How It Works



End-to-end pipeline transforms raw GitHub data into actionable risk scores.

Key Finding: Popularity ≠ Maintenance



Backlog Warning

These "healthy" projects have **100+ open issues** — a hidden maintenance risk.

📄 **Correlation: -0.72** between stars and maintenance. Famous ≠ well-maintained.

📄 **Team Size Impact:** Larger teams close issues **3x faster** than solo maintainers.

Portfolio Health: 10 Projects Analyzed

77.3

Overall Score

Out of 100 — Good portfolio health

27.0

Activity

Out of 30 — Very active

24.0

Team Size

Out of 30 — Large teams

12.8

Response Speed

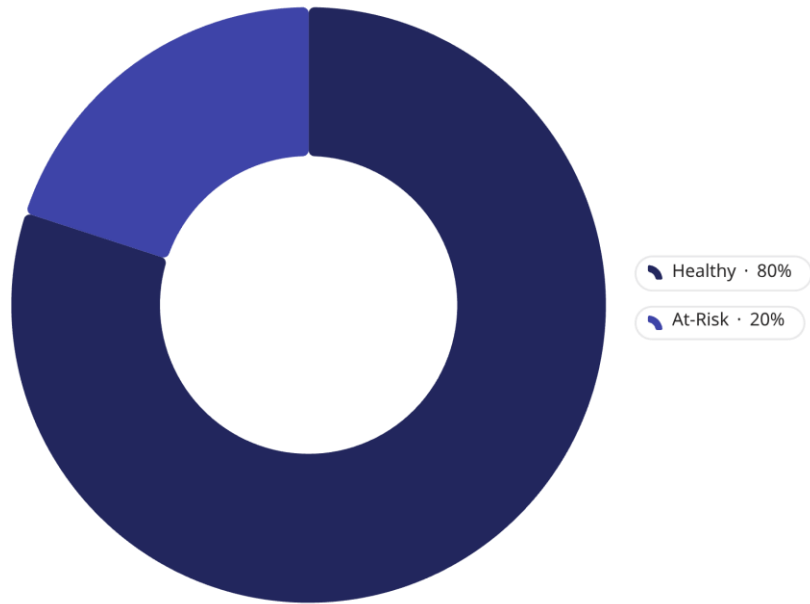
Out of 25 — Moderate backlog

13.5

Consistency

Out of 15 — Regular updates

Results at a Glance



10 Major Projects

Analyzed across 4 health dimensions using real GitHub data.

- **8 projects** marked healthy
- **2 projects** flagged at-risk
- Identified projects with **100K+ stars** but low maintenance signals

Tech Stack



SQL

Store and organize raw GitHub data



Python

Calculate health scores across 4 dimensions



Power BI

Interactive visualization and reporting

Lessons Learned

Data Quality > Data Volume

React scored 0 due to API rate-limits — completeness beats scale

Correlation $\neq$ Causation

Stars and health correlate at -0.72 , partly due to API limits

Scoring Weights Matter

Activity weighted 30% vs. stability 15% — reflects business impact

API Constraints Are Real

GitHub limits 5K calls/hour — large repos need special handling

Limitations & Path Forward

Known Limitations

- **Rate-limits** cap commits for large repos
- **6-month window** misses long-term trends
- **Open issues** skew responsiveness scores

Recommended Fixes

- Store **historical data** for trend tracking
- Fetch more **closed issues** for accuracy
- Adjust **weighting** based on data quality



Next step: Expand to 6-month rolling analysis with historical data storage.